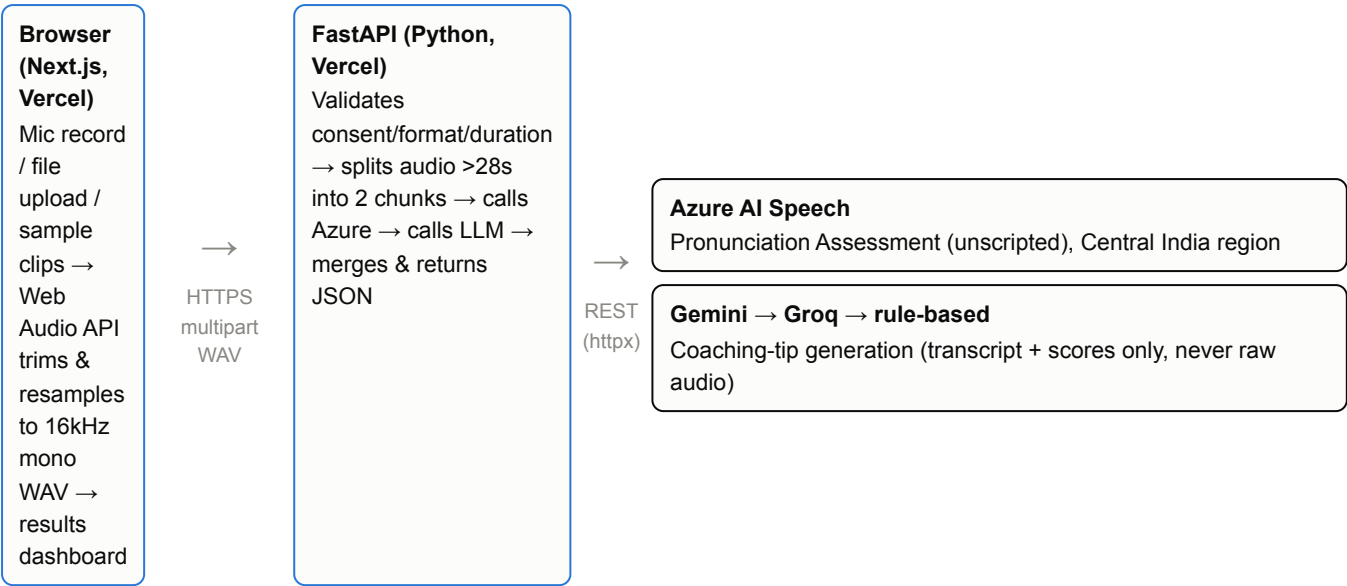


PronounceRight — System Architecture

Livo AI Software Engineer Assessment · K Ashraf Ahmed · July 2026

1. Overview & component diagram

A learner uploads or records 30–45s of English speech in the browser, which transcodes it to 16kHz mono WAV client-side. A FastAPI backend runs Azure AI Speech's Pronunciation Assessment for phoneme-level scoring, then an LLM turns the raw scores into plain-language coaching tips. Nothing is persisted server-side.



2. Components & why

Piece	Choice	Why over the alternative
Scoring engine	Azure AI Speech Pronunciation Assessment	Purpose-built GOP-based phoneme scoring with an error-type taxonomy; free F0 tier. A DIY Whisper+wav2vec2+G2P stack needs GPU hosting and gives noisier scores; an LLM alone can't reliably judge phoneme accuracy by ear.
Speech API style	Plain REST (httpx)	The Speech SDK is a native binary — heavier serverless bundle, slower cold starts. Plain HTTP POST is retryable and has no persistent-connection failure modes.
Long-audio handling	Split into 2 sub-30s chunks, call in parallel	Azure's short-audio endpoint caps pronunciation assessment at ~30s; our required range is 30–45s. Rather than depend on the SDK's stateful "continuous mode" streaming session inside a serverless function, we split each clip at its quietest point near the midpoint (found by scanning short-window RMS energy) and merge the two results, weighting full-

4. DPDP Act 2023 compliance

- **Consent:** explicit checkbox required before any upload; backend rejects requests without it.
- **Storage/retention:** audio is held in memory only for the single request that scores it — never written to disk, a database, or object storage. The backend retains nothing once the response is sent.
- **Data residency:** Azure Speech resource provisioned in Central India. Coaching generation sends only the transcript + numeric scores (never raw audio) to Gemini/Groq, which may process outside India — permitted under DPDP §16 absent a government restriction on that destination.
- **Deletion:** nothing to delete server-side by construction. Client keeps the recording in memory only until "Analyze another recording" or the tab closes.
- **Rights & grievance:** a dedicated in-app privacy notice covers access/correction/erasure rights and a grievance contact, per §11–13.

5. Reliability (evaluator must never see a cold/broken app)

- Vercel (not Render) — no 15-min idle sleep; worst case is a ~1s cold start on a tiny httpx-only bundle.
- External keep-alive pinger hits `/health` every 5 minutes, 24/7, keeping the function warm.
- Frontend fires a background `/health` ping on page load, so any residual cold start happens while the

utterance scores by chunk duration.

Coaching LLM	Gemini 2.5 Flash → Groq → rule-based	Three-tier fallback so a bad LLM day never breaks the request — scores and highlights always render even if every LLM call fails.
Audio transcoding	Client-side (Web Audio API)	Keeps uploads small (~1.4MB) and already-normalized; the backend never needs an audio library.
Hosting	Vercel (2 projects, 1 monorepo)	No idle "sleep" like some free tiers, matching the JD's own stack. Frontend and API are separate Vercel projects (each rooted at its own subfolder) to avoid mixed-runtime build issues.

3. Scoring & highlighting methodology

- **Sub-scores:** accuracy, fluency, prosody, completeness, and an aggregate *PronScore* returned directly by Azure; the UI shows all four plus the overall gauge.
- **Word band:** `good ≥80` `warning 60–79` `critical <60`, overridden to critical if Azure flags an explicit *Mispronunciation* error type regardless of score.
- **Phoneme drill-down:** clicking a word reveals its phoneme-level accuracy bars (IPA symbols) — this is the "what specifically went wrong" layer the assignment asks for, one level below the word heatmap.
- **Unscripted mode** (empty reference text) was chosen over scripted reading so the tool works on any free-form speech, not just a fixed passage — matching a real learner's use case. The trade-off: without a reference transcript, Azure can't tag Omission/Insertion errors (those require a known reference), so v1 surfaces mispronunciation and low-accuracy words but not skipped/extra words. See trade-offs.

evaluator is still reading, not when they click "Assess."

- `maxDuration=60s` on the `assess` function; graceful degradation if the LLM layer fails; every error path shows a friendly message, never a raw stack trace or blank hang.

6. Trade-offs & what's next

- **No Omission/Insertion detection** in v1 (see §3). Next: run a first STT pass to get a transcript, then a second scripted assessment pass using that transcript as reference text, enabling full miscue detection — at the cost of a second sequential API call.
- **Per-IP rate limiting is in-memory**, best-effort only — resets on cold start, not shared across instances. Next: Redis-backed limiter (e.g. Upstash) for real multi-instance enforcement.
- **No accounts or history.** Next: optional sign-in + a Postgres table of past attempts with a short retention TTL, so a learner can track improvement over time (would need its own consent/retention story).
- **Single-language (en-US).** Azure supports pronunciation assessment in other locales; multi-accent reference support (en-IN, en-GB) would make scoring feel fairer to non-US speakers.
- **Eval set is small** (2 labeled clips checked for score consistency in `evals/`). Next: a larger human-rated set with inter-rater agreement, to validate the accuracy-score thresholds against real learner judgment rather than Azure's raw output.